

Flipkart Customer Support Analysis

Project Type -Exploratory Data Analysis (EDA)

Contribution - Individual Level

Name - Mannepalli Abhilash

Project Summary

In the fiercely competitive world of e-commerce, maintaining growth and cultivating client loyalty depend heavily on providing exceptional customer service. In order to determine the primary factors influencing customer satisfaction, this project will make use of Flipkart's sizable dataset of customer interactions, reviews, and satisfaction ratings. We will be able to identify areas for strategic improvement and obtain a thorough grasp of performance across different customer service teams by examining these important data points. The ultimate objective is to create and put into action focused strategies that improve the overall customer experience and guarantee Flipkart's continued dominance in customer-focused e-commerce.

A deep understanding of the factors influencing customer satisfaction will empower Flipkart to not only streamline issue resolution processes but also to customize its support strategies to precisely meet the diverse expectations of its vast customer base. This data-driven approach will be instrumental in optimizing the efficiency and effectiveness of service agents, directly contributing to a significant improvement in crucial satisfaction metrics like the CSAT score. Ultimately, these enhancements are designed to cultivate stronger brand loyalty and ensure long-term customer retention, reinforcing Flipkart's market leadership.

Problem Statement

Flipkart constantly has to stand out from the competition in the fiercely competitive e-commerce market and cultivate enduring client loyalty by providing exceptional customer service. Even though there are many customer interactions, it is imperative to thoroughly examine them in order to pinpoint the fundamental causes of both customer satisfaction and discontent. Flipkart runs the risk of inefficient resource allocation, uneven service quality, and lost opportunities to greatly improve its Customer Satisfaction (CSAT) scores, all of which will have an adverse effect on customer retention and brand loyalty, if these factors and the performance variances across various customer service channels and teams are not well understood.

Key Areas of Focus:

- > Proactive Service Strategy and Feedback Loop Integration
- > Customer Service Performance Evaluation
- > Proactive Service Strategy and Feedback Loop Integration
- > Strategic Recommendation Development
- > Channel and Issue Category Optimization
- > CSAT (Customer Satisfaction Source)

```
# Importing Important Libraries.
import pandas as pd
import numpy as np
import sys
import matplotlib.pyplot as plt
from matplotlib import colormaps
import seaborn as sns
from IPython.display import display, HTML

# Importing Warning Library in order to avoid unnecessary warning.
import warnings
warnings.filterwarnings("ignore")
```

```
# Loading the Data Set

DataSet = "Customer_support_data.csv"
dataset = r"/Users/abhillaahsh/Downloads/Flipkart project/
Customer_support_data.csv"
loaded_csv = pd.read_csv(DataSet)
print(loaded_csv)

# sys.exit()
```

```
# display(HTML("<b>dataframe shape:</b>"))

print(f"shape:{loaded_csv.shape}\n")
```

shape:(85907, 20)

```
# display(HTML("<b>dataframe information:</b>"))

print(f"{loaded_csv.info()}\n")
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 85907 entries, 0 to 85906
```

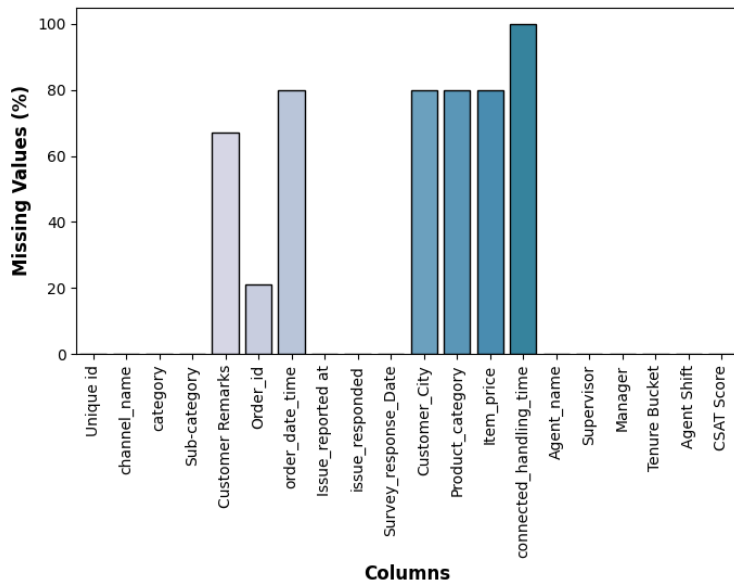
```
Data columns (total 20 columns):
```

#	Column	Non-Null Count	Dtype
0	Unique id	85907 non-null	object
1	channel_name	85907 non-null	object
2	category	85907 non-null	object
3	Sub-category	85907 non-null	object
4	Customer Remarks	28742 non-null	object
5	Order_id	67675 non-null	object
6	order_date_time	17214 non-null	object
7	Issue_reported at	85907 non-null	object
8	issue_responded	85907 non-null	object
9	Survey_response_Date	85907 non-null	object
10	Customer_City	17079 non-null	object
11	Product_category	17196 non-null	object
12	Item_price	17206 non-null	float64
13	connected_handling_time	242 non-null	float64
14	Agent_name	85907 non-null	object
15	Supervisor	85907 non-null	object
16	Manager	85907 non-null	object
17	Tenure Bucket	85907 non-null	object
18	Agent Shift	85907 non-null	object
19	CSAT Score	85907 non-null	int64

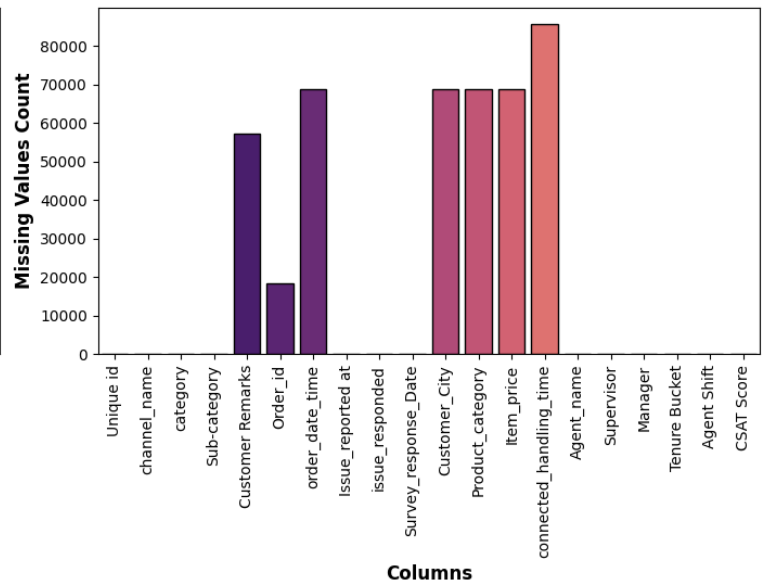
```
dtypes: float64(2), int64(1), object(17)
```

```
NullValues = loaded_csv.isnull().sum()
# Null Value Percentage Present
NV_Percentage = round(loaded_csv.isnull().sum()/len(loaded_csv)*100,0)
print ("\n")
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
# chart-1 shows null values in %.
sns.barplot(x=NV_Percentage.index, y=NV_Percentage.values,
edgecolor="black", linewidth=1, palette='PuBuGn', ax=axes[0] )
axes[0].set_xlabel("Columns", fontsize=12, fontweight='bold')
axes[0].set_ylabel("Missing Values (%)", fontsize=12, fontweight='bold')
axes[0].set_xticklabels(NV_Percentage.index, rotation=90)
axes[0].set_title("Percentage of Missing Values in Each
Column", fontweight='bold'
, pad =20)
# chart-2 Null Values Distribution Across Columns.
sns.barplot(x=NullValues.index, y=NullValues.values, edgecolor="black",
linewidth=1, palette='magma', ax=axes [1] )
axes [1].set_xlabel("Columns", fontsize=12, fontweight='bold')
axes [1].set_ylabel("Missing Values Count", fontsize=12, fontweight='bold')
axes[1].set_xticklabels(NV_Percentage.index, rotation=90)
axes [1].set_title("Null Values Distribution Across
Columns", fontweight='bold', pad =20)
plt. tight_layout()
plt. show()
print ("\n")
```

Percentage of Missing Values in Each Column



Null Values Distribution Across Columns



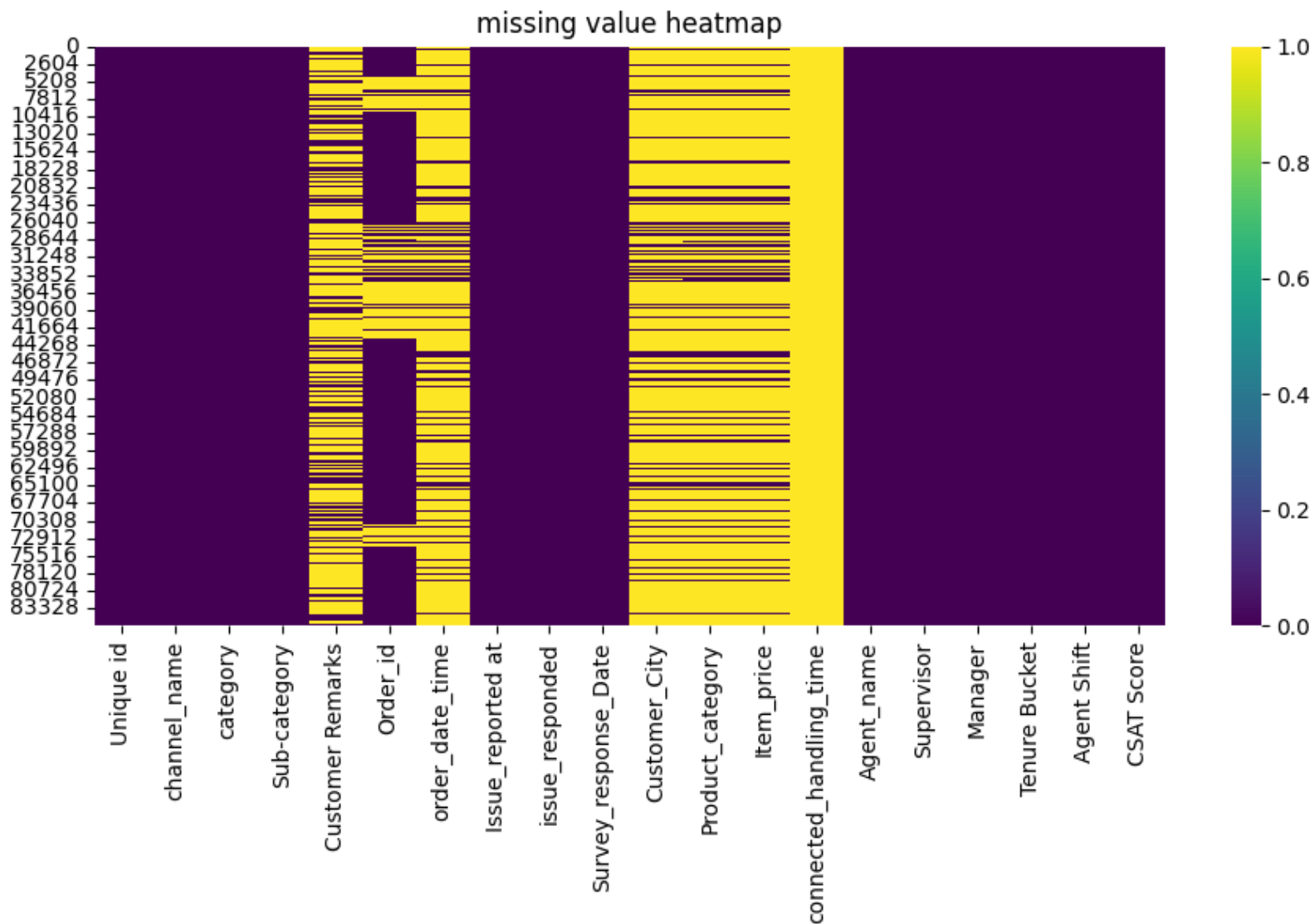
Information

- > Number of Rows: 85,907
- > Number of Columns: 20
- > Duplicate ROWS: 0 (No duplicate records)
- > Column Data Types: Mostly object (text) data (e.g., Unique ID, category, agent names) and Some numerical columns (e.g., Item Price, CSAT Score)

Observation

- > Dataset has no duplicate records
- > Missing data in core order fields is problematic because these are foundational for many business analyses (Order ID, Date, Product Category)
- > The CSAT Score column (CUSTOMER SATISFACTION) is fully available.
- > Columns with such a high proportion of missing data (more than 70%) are likely to be of limited analytical value

```
# 1. missing value heatmap
print("\n")
plt.figure(figsize=(10, 6))
sns.heatmap(loader.csv.isnull(), cmap='viridis', cbar=True)
plt.title("missing value heatmap")
plt.show()
print("\n")
```



Observation:

- > Missing values in customer remark, order date time, customer city, product category, and connected handling time may impact data analysis, and decision making
- > Complete data and key columns by issue reported at survey response date ,agent name, and CSAT score allows for more meaningful insights into customer satisfaction and the agent performance.
- > handling missing data thought imputation, removal, or investigation crucial to improve the accuracy and reliability of customers support analysis

```
# Data Cleaning Steps
# 1. Removing all the Duplicates.
loaded_csv_cleaned = loaded_csv.drop_duplicates()
# 2. Replacing Null values with Default values.
loaded_csv_cleaned.fillna({ "Customer Remarks": "No Feedback Provided",
                           "Customer_City": "Not Mentioned",
                           "Product_category" : "Not Available",
                           "Item_price" :
loaded_csv_cleaned["Item_price"].median(),
                           "Order_id" : "Not Available"
                           },inplace = True)
```

```

# Data Cleaning Steps
# 1. Removing all the Duplicates.
loaded_csv_cleaned = loaded_csv.drop_duplicates()
# 2. Replacing Null values with Default values.
loaded_csv_cleaned.fillna({ "Customer Remarks": "No Feedback Provided",
                             "Customer_City": "Not Mentioned",
                             "Product_category" : "Not Available",
                             "Item_price" :
loaded_csv_cleaned["Item_price"].median(),
                             "Order_id" : "Not Available"
                             },inplace = True)

# 3. Deleting the Unnecessary Columns
loaded_csv_cleaned = loaded_csv_cleaned.drop(columns = ["order_date_time",
"connected_handling_time"])
# 4. Data Standarization
# Converting the columns into Date data type
DateTime = ['Issue_reported at', 'issue_responded', 'Survey_response_Date']
for col in DateTime:
    loaded_csv_cleaned[col] =
pd.to_datetime(loaded_csv_cleaned[col],dayfirst= True, errors="coerce")

# changing columns name
loaded_csv_cleaned = loaded_csv_cleaned.rename(columns = {"channel_name":
"Channel Name",
                 "category": "Category",
                 "Order_id": "Order id",
                 "Issue_reported at" : "Issue Reported At",
                 "issue_responded": "Issue Responded",
                 "Survey_response_Date": "Survey Response Date",
                 "Customer_City" : "Customer City",
                 "Product_category" : "Product Category",
                 "Item_price": "Item Price",
                 "Agent_name" : "Agent Name"
                 })
# setting column(Unique id) as index values.
loaded_csv_cleaned = loaded_csv_cleaned.set_index("Unique id")
loaded_csv_cleaned ["Customer Remarks"] = loaded_csv_cleaned["Customer
Remarks"].str.rstrip()

```

```
# Important Information related to Dataframe.
# Dataset Columns
# display(HTML("<b>Columns of the Data set:</b>"))
print(f"{loaded_csv_cleaned.columns}\n\n")
```

```
index(['Channel Name', 'Category', 'Sub-category', 'Customer Remarks',
       'Order id', 'Issue Reported At', 'Issue Responded',
       'Survey Response Date', 'Customer City', 'Product Category',
       'Item Price', 'Agent Name', 'Supervisor', 'Manager', 'Tenure Bucket',
       'Agent Shift', 'CSAT Score'],
      dtype='object')
```

```
# Dataset Description
# display(HTML("<b>Important information about the Dataset:</b>"))
loaded_csv_cleaned.describe()
```

	Issue Reported At	...	CSAT Score
count	85907	...	85907.000000
mean	2023-08-16 22:29:05.784394752	...	4.242157
min	2023-07-28 20:42:00	...	1.000000
25%	2023-08-09 12:48:00	...	4.000000
50%	2023-08-16 18:22:00	...	5.000000
75%	2023-08-24 17:33:30	...	5.000000
max	2023-08-31 23:58:00	...	5.000000
std	NaN	...	1.378900

```
# Check Unique Values for each variable.
unique_values = loaded_csv_cleaned.nunique()
# display(HTML("<b>Unique Values:</b>"))
print(f"{unique_values}\n\n")
```

Category	12
Sub-category	57
Customer Remarks	17702
Order id	67676
Issue Reported At	30923
Issue Responded	30262
Survey Response Date	31
Customer City	1782
Product Category	10
Item Price	2789
Agent Name	1371
Supervisor	40
Manager	6
Tenure Bucket	5
Agent Shift	5
CSAT Score	5

dtype: int64

```
# Top 5 Rowa of the Data set
print(loaded_csv_cleaned.head())
```

Unique id	Channel Name	...	CSAT Score
7e9ae164-6a8b-4521-a2d4-58f7c9fff13f	Outcall	...	5
b07ec1b0-f376-43b6-86df-ec03da3b2e16	Outcall	...	5
200814dd-27c7-4149-ba2b-bd3af3092880	Inbound.	...	5
eb0d3e53-c1ca-42d3-8486-e42c8d622135	Inbound	...	5
ba903143-1e54-406c-b969-46c52f92e5df	Inbound	...	5

[5 rows x 17 columns]

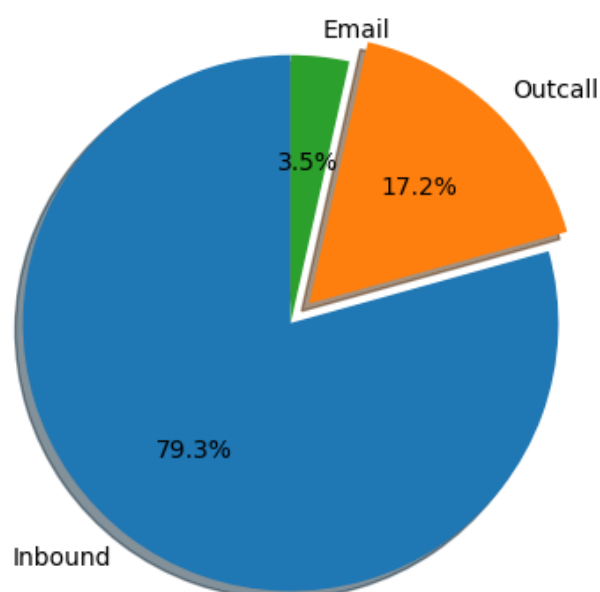

```

# Univariate Analysis.
# Chart No.1 – Pie Chart

ChannelName = loaded_csv_cleaned ["Channel Name"].value_counts().index
Channelcount = loaded_csv_cleaned ["Channel Name"].value_counts().values
explode = (0,0.1,0)
print ("\n")
fig,ax = plt.subplots(figsize=(12, 4))
ax.set_position([0.2, 0.2, 0.6, 0.6])
ax.pie(Channelcount, labels=ChannelName, autopct='%1.1f%%' ,
        startangle=90, explode=explode, shadow = True)
ax.axis("equal")
plt.title("Univariate Analysis\n Channel-Wise Breakdown of Resolved Queries",
fontweight='bold', pad= 20)
plt.tight_layout()
plt.show()
print("\n")

```

Channel-Wise Breakdown of Resolved Queries



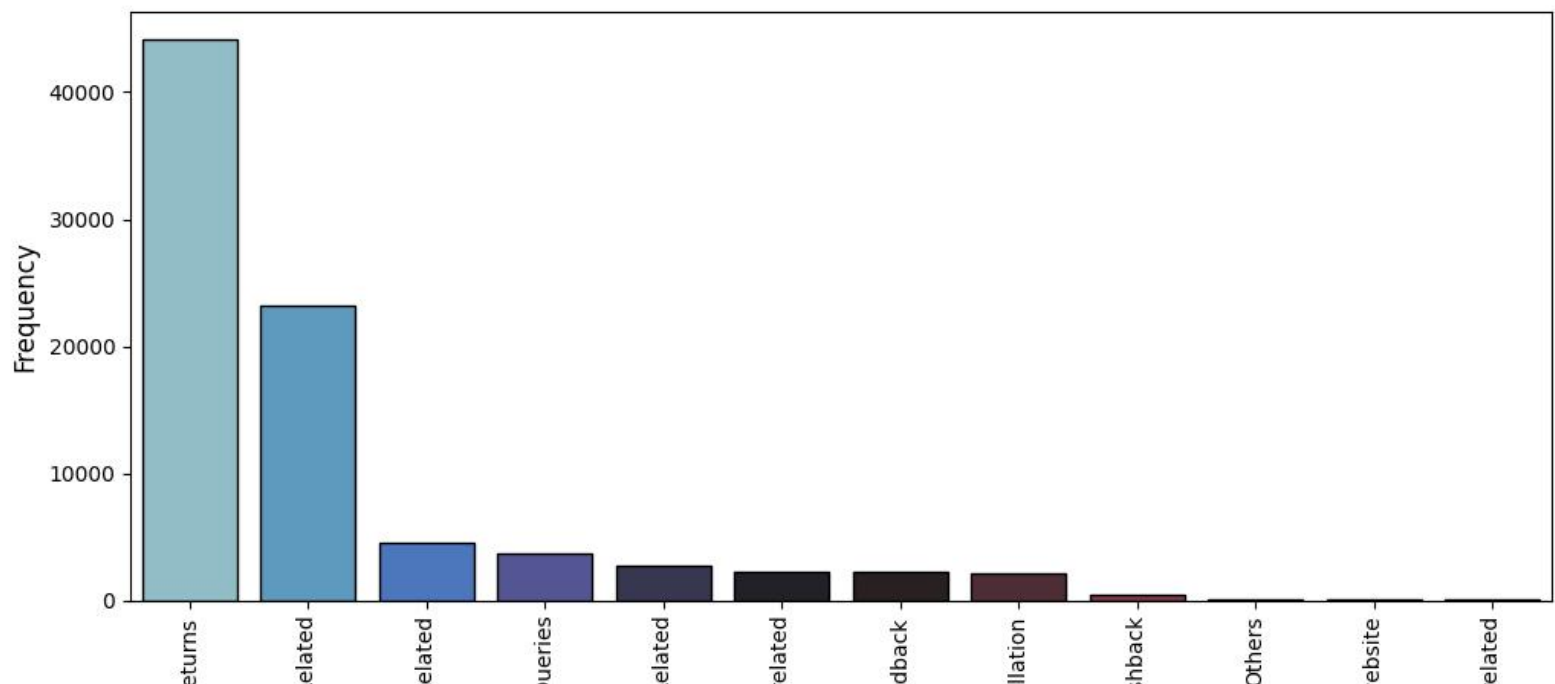
Observation:

- > **Inbound queries dominate** customer support, resolving **79.3%** of issues.
- > **Outcall support handles 17.2%**, likely for follow-ups or proactive outreach.
- > **Email support is minimal**, contributing only **3.5%** to resolved queries.

```
# Univariate Analysis
Category = loaded_csv_cleaned["Category"].value_counts().index
CategoryValues = loaded_csv_cleaned["Category"].value_counts().values

# Chart No.2 - Bar Chart
print("\n" )
fig,ax = plt.subplots (figsize=(12, 5))
sns.barplot(x = Category, y=CategoryValues,edgecolor="black",
linewidth=1, palette = "icefire")
ax.set_xlabel("Category Type", fontsize=12)
ax.set_ylabel("Frequency", fontsize=12)
ax.set_xticklabels(Category, rotation=90)
ax.set_title("Frequency Distribution of Categories",fontweight='bold',
pad =20)
plt. show()
print("\n")
```

Frequency Distribution of Categories



Observation:

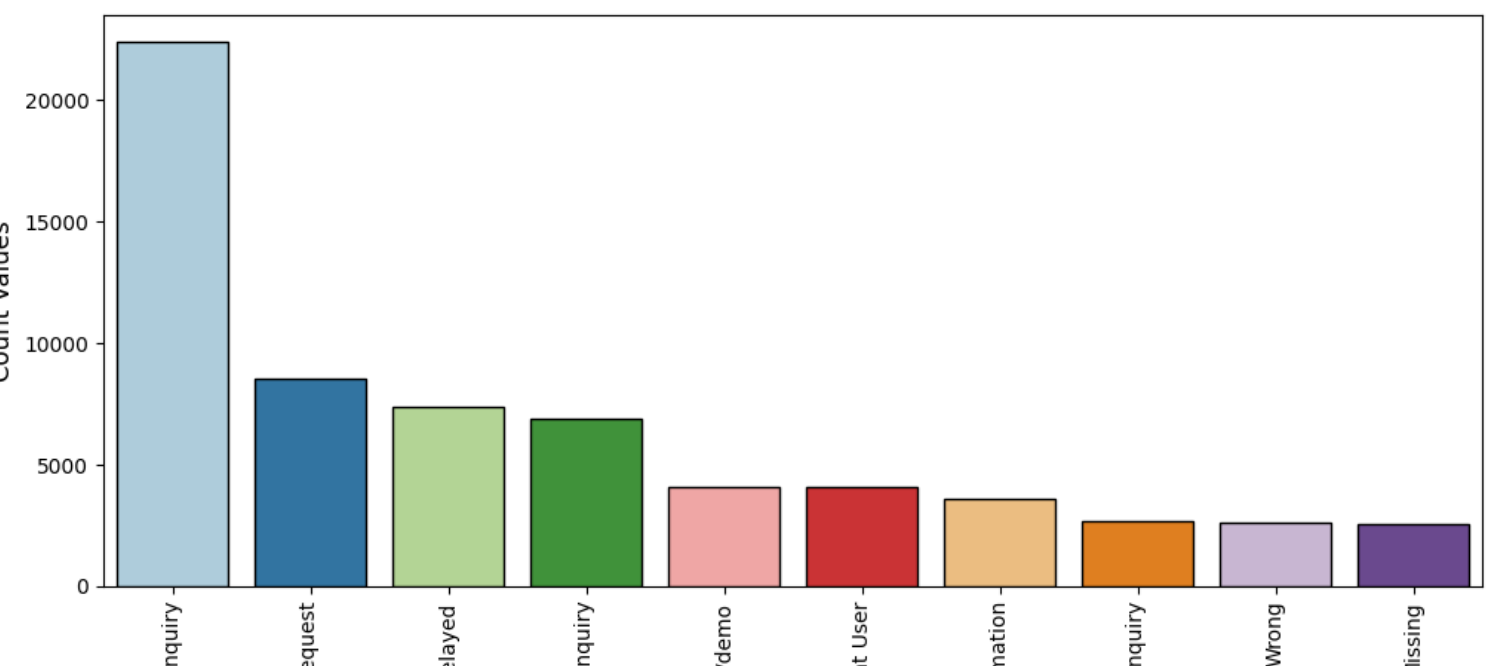
- > **Returns** are the most frequent customer support queries.
- > **Order and refund** issues are the next most common concerns.
- > Queries about **onboarding, website issues, and offers** are minimal.

Univariate Analysis

```
SubCategory = loaded_csv_cleaned["Sub-category"].value_counts().index
SubCategoryValues = loaded_csv_cleaned["Sub-category"].value_counts().values
# Chart No.3 – Bar Chart

print("\n")
fig,ax = plt.subplots(figsize=(12, 5))
sns.barplot(x = SubCategory[:10], y=SubCategoryValues[:10],edgecolor="black",
linewidth=1, palette = "Paired")
ax.set_xlabel("Columns", fontsize=12)
ax.set_ylabel("Count Values", fontsize=12)
ax.set_xticklabels(SubCategory, rotation=90)
ax.set_title("Different types of sub-Category Count
Frequency",fontweight='bold', pad =20)
plt.show()
print("\n")
```

Different types of sub-Category Count Frequency



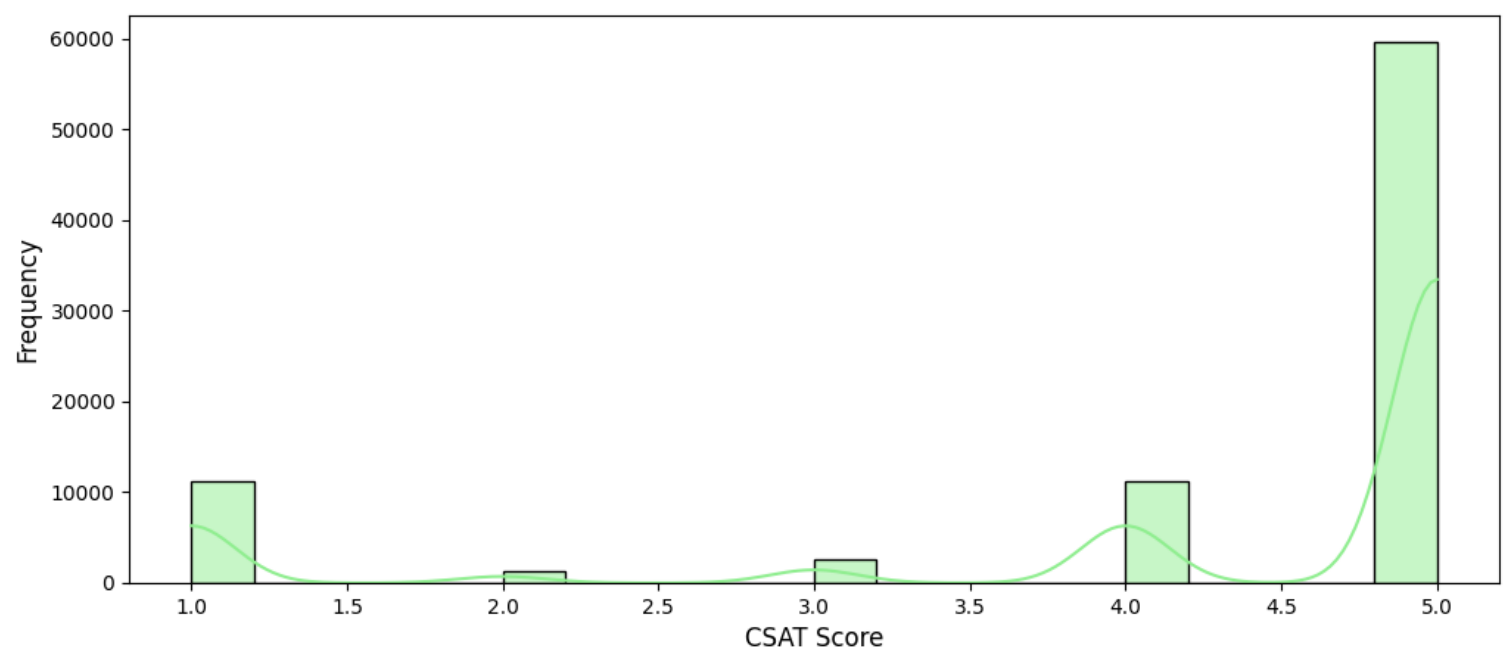
Observation:

- > **Reverse Pickup Enquiries** are the most common customer support sub-category, showing a high volume of return-related requests.
- > **Return Requests** and **Delayed Orders** are significant issues, suggesting a need for better logistics and order fulfillment.
- > **Fraudulent User** and **Product-Specific Information** queries are also frequent, pointing to concerns with customer verification or product details.
- > **Refund Enquiry**, **Wrong Item**, and **Missing Product** issues occur less frequently but still impact customer satisfaction and service efficiency.

Chart No.4 – Bar Chart

```
print ("\n")
fig,ax = plt.subplots(figsize=(12, 5))
sns.histplot(x = "CSAT Score", data = loaded_csv_cleaned ,bins =
15,binwidth=0.2, kde = True, edgecolor="black",linewidth=1,
color='lightgreen')
ax.set_xlabel("CSAT Score",fontsize=12)
ax.set_ylabel ("Frequency", fontsize=12)
ax.set_title("Customer Satisfaction Score
Distribution",fontweight='bold', pad =20)
plt.show()
print("\n")
```

Customer Satisfaction Score Distribution



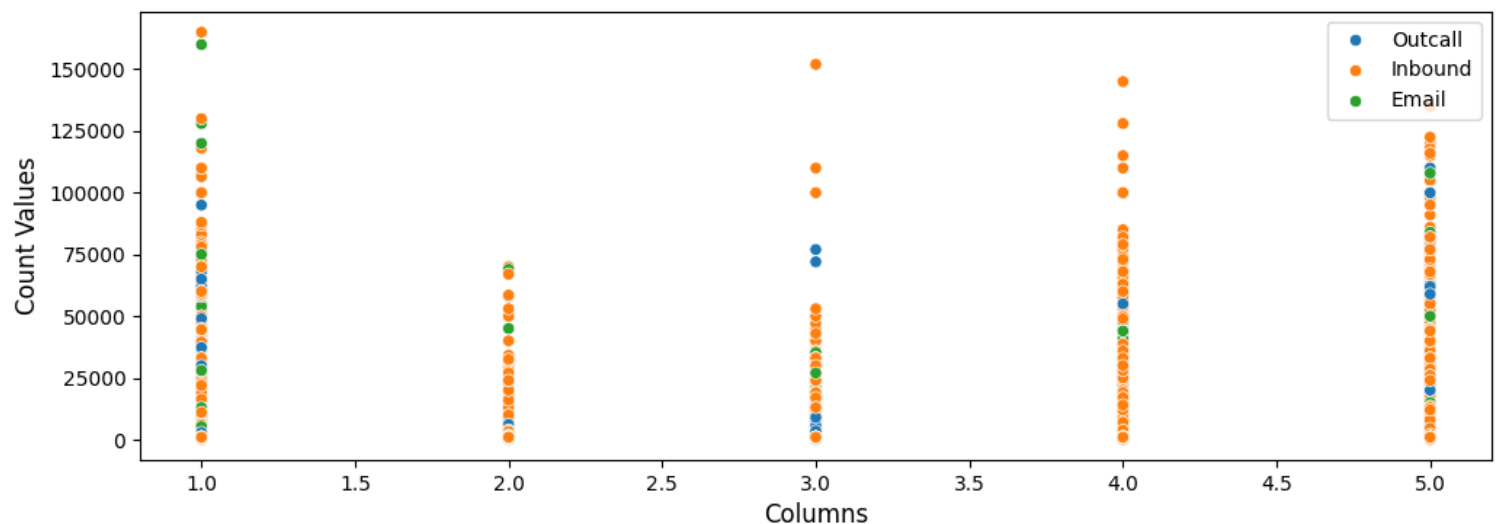
Observation:

- > Customer satisfaction is **polarized**, with most ratings at **5.0 (highly satisfied)** and a smaller peak at **1.0 (highly dissatisfied)**.
- > There are **few neutral responses** (2.0 and 3.0), indicating customers are rarely ambivalent.
- > This suggests a customer experience that elicits strong feelings, either very positive or very negative.

Bivariate & Multivariate Analysis

```
print("\n")
fig, axes = plt.subplots(figsize=(12,4))
sns.scatterplot(x="CSAT Score", y="Item Price", hue="Channel Name",
data=loaded_csv_cleaned, palette='tab10')
axes.set_xlabel("Columns", fontsize=12)
axes.set_ylabel("Count Values", fontsize=12)
axes.set_title("Distribution : CSAT Score VS Item
Price", fontweight='bold', pad =20)
plt.legend (loc="upper right")
plt.show()
print("\n")
```

Distribution : CSAT Score VS Item Price



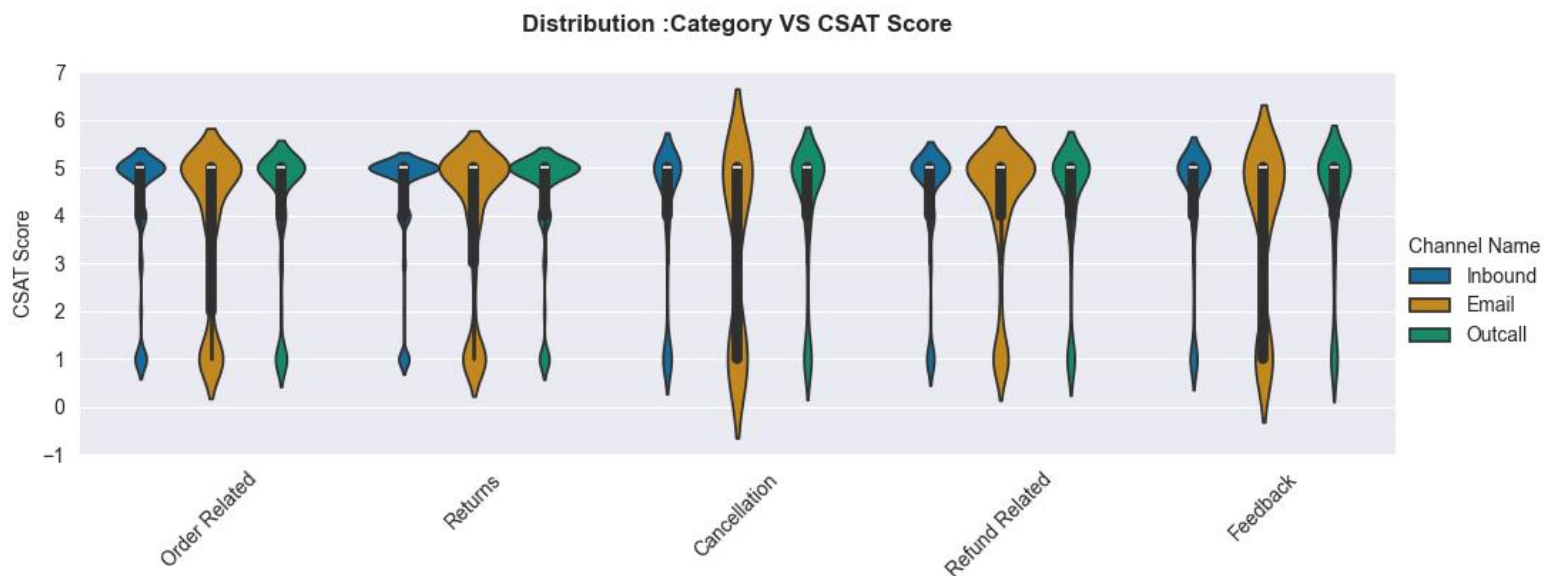
Observation:

- > **Reverse Pickup Enquiries** are the most frequent customer support sub-category, signaling high return-related requests.
- > **Return Requests** and **Delayed Orders** are significant concerns, pointing to logistics and fulfillment challenges.
- > **Fraudulent User** and **Product-Specific Information** issues also occur frequently, highlighting potential concerns with customer verification or product details.

```

CategoryData=
loaded_csv_cleaned[loaded_csv_cleaned["Category"].isin(['Returns', 'Refund
Related', 'Cancellation', 'Feedback', 'Order Related'])]
print ("\n")
sns.set_style("darkgrid")
sns.catplot(x='Category',y='CSAT Score', kind='violin',hue='Channel
Name' ,dodge =True, data=CategoryData,
            height=4, aspect=2.5, palette = "colorblind")
plt.xticks(rotation=45)
plt.title("Distribution :Category VS CSAT Score", fontweight='bold', pad
=20)
plt.show()
print ("\n")

```



Observation:

- > Customer satisfaction is **polarized**, with a large number of ratings at **5.0 (highly satisfied)** and a smaller but notable group at **1.0 (highly dissatisfied)**.
- > There are significantly **fewer middle-range scores** (2.0 and 3.0), indicating customers rarely feel neutral.
- > This suggests a **polarized customer experience**, where customers generally have strong positive or strong negative feelings.

```
# Bivariate & Multivariate Analysis
```

```
# CSAT Score vs. Issue Category
```

```
print("\n")
```

```
plt.figure(figsize=(13, 4))
```

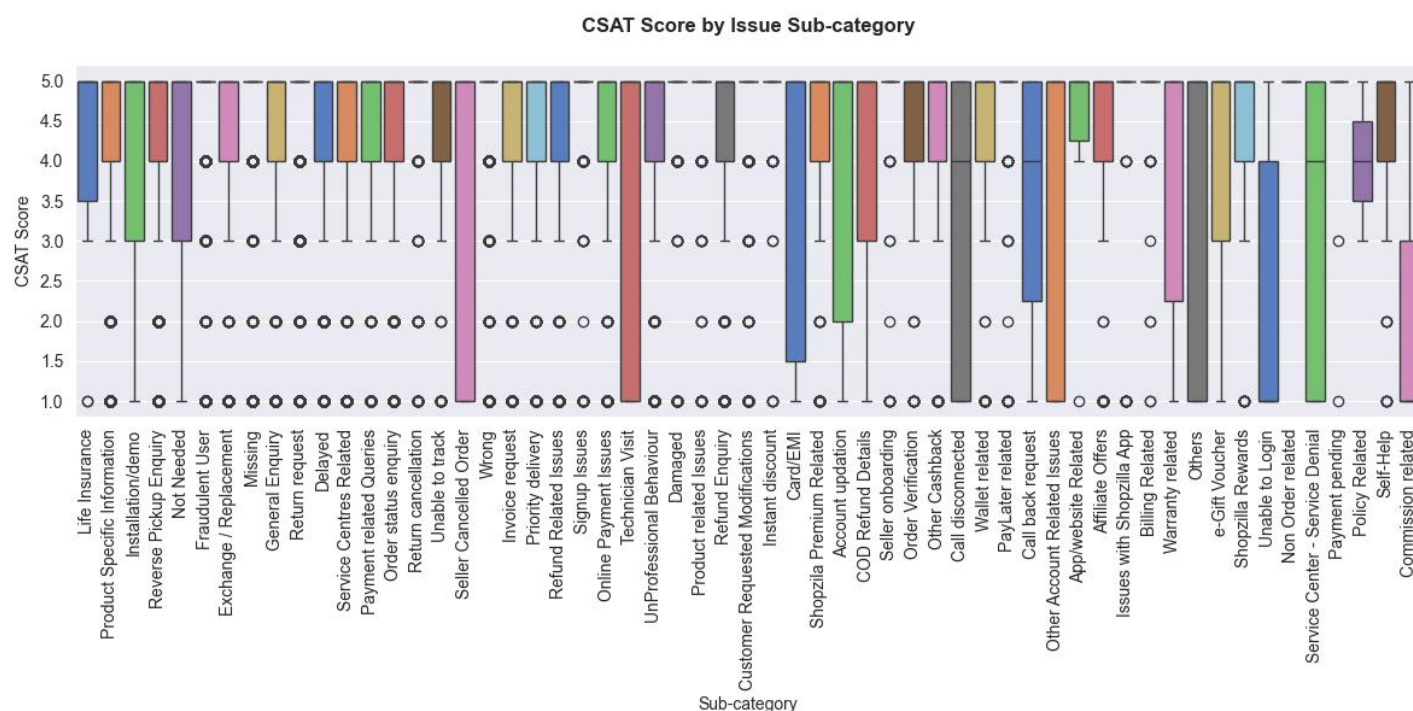
```
sns.boxplot(x='Sub-category', y='CSAT Score', data=loaded_csv_cleaned, palette='muted')
```

```
plt.xticks(rotation=90)
```

```
plt.title("CSAT Score by Issue Sub-category", fontweight='bold', pad =20)
```

```
plt.show()
```

```
print("\n")
```

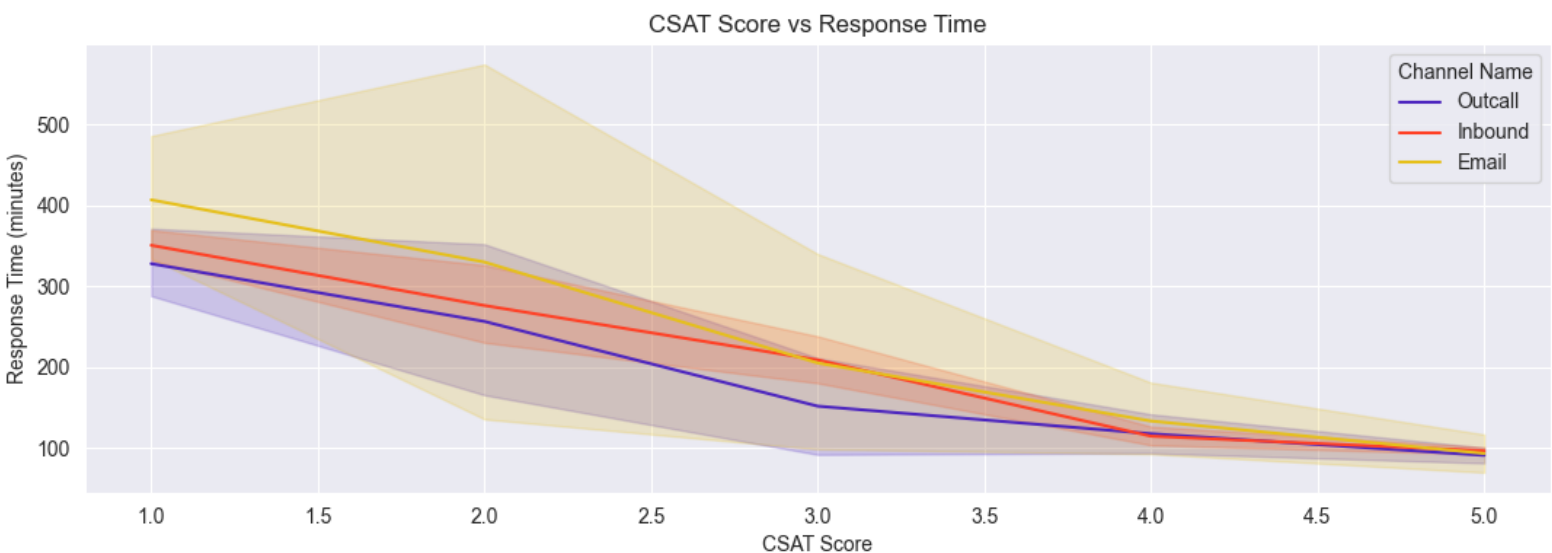


Observation:

- > **"Cancellation" and "Returns"** show **mixed CSAT scores**, with both high and low ratings.
- > **"Life Insurance" and "Product Specific Information"** consistently receive **high CSAT scores**.
- > **Outliers of low scores** appear across multiple categories, indicating isolated poor experiences.

```
# calculating Response time in Minutes.
RP_time= loaded_csv_cleaned[ 'Issue Responded'] - loaded_csv_cleaned[ 'Issue
Reported At']
RP_time = RP_time.dt.total_seconds() / 60

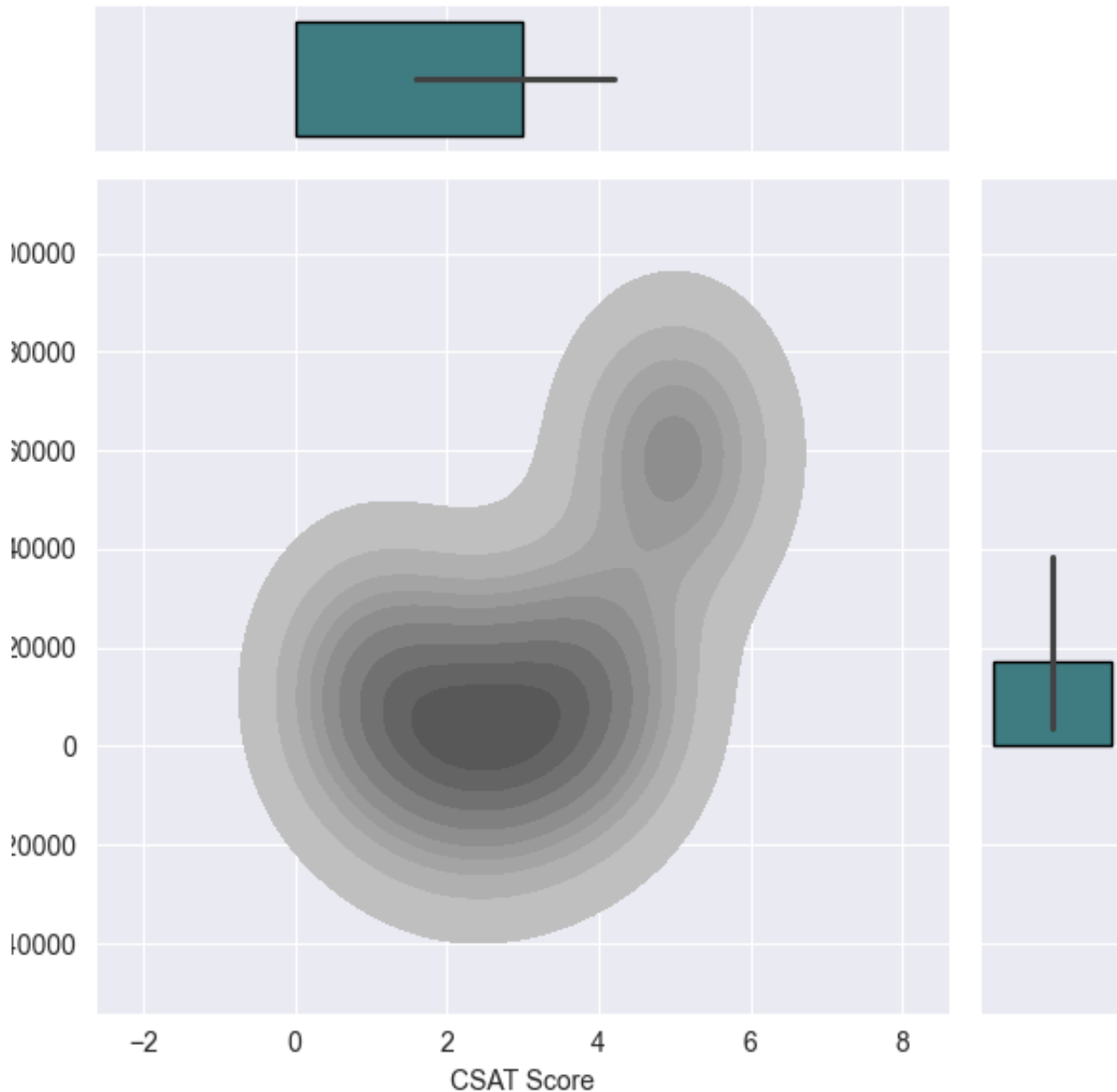
print ("\n")
fig, axes = plt.subplots(figsize=(13,4))
sns.lineplot(x='CSAT Score', y=RP_time, data=loaded_csv_cleaned,
hue='Channel Name', palette="CMRmap", ax=axes)
axes.set_xlabel("CSAT Score")
axes.set_ylabel ("Response Time (minutes)")
axes.set_title("CSAT Score vs Response Time")
plt.show()
print("\n")
```



Observation:

- > **Faster responses correlate with higher CSAT scores** across all channels.
- > **Email has the most variable response times**, potentially leading to inconsistent customer satisfaction.
- > **Outcall support offers the quickest responses for low CSAT scores**, suggesting its effectiveness for urgent issues.


```
print ("\n" )
g = sns.JointGrid(x=loaded_csv_cleaned["CSAT Score"].value_counts().index,
y =loaded_csv_cleaned ["CSAT Score"].value_counts().values)
g.plot(sns.kdeplot, sns.barplot, edgecolor = "black", fill = True, palette
= "crest_r", color = "grey")
g.set_axis_labels (xlabel = "CSAT Score", ylabel = "Frequency")
plt.show()
print("\n")
```



Observation:

- > Most CSAT scores fall between 2 and 5, with the highest frequency around 3-4.
- > Outliers exist at negative and very high values, potentially indicating data errors.
- > Higher CSAT scores are more frequent, suggesting feedback leans towards satisfaction.

Enhancing the Flipkart customer support

1. **Improve customer satisfaction (CSAT Score):**

- > Consistent "Cancellations" and "Returns" experiences result in varying levels of satisfaction.
- > The initial customer effort is concealed by a high CSAT for clear information ("Life Insurance," "Product Specific Information").
- > Outliers with low scores suggest isolated service issues that require attention.

Proposal:

- > Examine low CSAT scores (1.0) to identify and address systemic problems rather than isolated ones.
- > Establish a strong feedback loop amongst teams to enhance operations, policies, and products over time.
- > Encourage a customer-centric culture among agents by rewarding high CSAT performance.

2. **Dominance of Inbound Queries and Channel Performance Discrepancies:**

- > First-contact resolution relies heavily on inbound queries, which account for 79.3% of all queries.
- > Outcall support (17.2%) is valuable for proactive engagement, especially for urgent issues and low CSAT.
- > Email support (3.5%) is minimal and inconsistent, leading to unpredictable customer satisfaction.

Proposal:

- > Reduce incoming inquiries by streamlining "Reverse Pickup" processes with automation and self-service.
- > In order to control expectations and provide solutions, implement proactive notifications for "Delayed Orders."
- > Implement tiered handling for "Return Requests," automating straightforward situations and sending more complicated ones to qualified agents.

3. Polarized Customer Satisfaction: A Clear Signal:

- > With few neutral experiences and a notable peak at 1.0 (dissatisfied) and the majority of ratings at 5.0 (satisfied), CSAT is extremely polarized.
- > Particularly in high-stress categories, the absence of middle-range scores (2.0–3.0) indicates that customer experiences are either very good or very bad.
- > The persistent 1.0 peak is a serious concern, and extreme outliers may indicate data problems, even though satisfaction typically skews moderate to high.

Proposal:

- > Reduce variability by optimizing email response times through AI triage, intelligent routing, and dynamic staffing.
- > Extend data-driven proactive outcall tactics to find and address possible customer concerns before they become more serious.
- > To improve empathy, conflict resolution, and First Contact Resolution across all channels, provide agents with unified desktops and continual training.

conclusion

Finally, our thorough analysis of Flipkart's customer service data reveals a crucial directive: to adopt a deeply customer-centric culture and go beyond transactional problem-solving. Although many customers report high levels of satisfaction, the sharp differences in CSAT scores and the enduring nature of persistent problems highlight the urgent need for decisive, strategic action to raise the bar for the customer experience. This change, which is infused with human insight, is expected to create stronger bonds and lead to revolutionary advancements.